BenchScout: Accelerating System Benchmarking with a Robust Machine Learning Pipeline

ABSTRACT

Benchmark suites are vital for evaluating computing platforms, but their comprehensive, all-or-nothing execution model incurs prohibitive time and resource costs. While replacing physical execution with machine learning (ML) predictions is a tempting alternative, applying naïve ML directly to system benchmarking fails in practice due to highly dynamic runtime contention, inflexible run-to-completion workloads, and inherently imperfect production telemetry. We present BenchScout, a systems toolchain that *accelerates* repeated execution of standard suites by keeping sparse real measurements in the loop: it selects a small informative subset, observes each component through a bounded probe, and reconstructs the official suite profile. BenchScout targets iterative tuning and regression workflows that need the same suite indices at lower cost; it is not a substitute for certified run-to-completion submissions where auditability requires verbatim protocol execution. Inspired by Mixture-of-Experts routing, BenchScout collects live system context, runs time-bounded probes on Top-K components, and reconstructs the remaining scores. Execution anomalies and missing telemetry are encoded as features rather than hard failures. Across heterogeneous hosts, BenchScout recovers aggregate suite scores within sub-percent error while reducing wall-clock time by two orders of magnitude.

CCS CONCEPTS

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

KEYWORDS

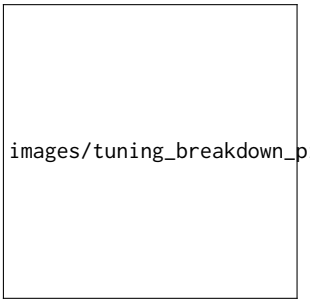
Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

. 2018. BenchScout: Accelerating System Benchmarking with a Robust Machine Learning Pipeline. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Benchmark suites [**? ? ?**] characterize modern computing platforms by running standardized workloads across processors, memory, storage, system software, and accelerators to produce a repeatable performance profile beyond any single measurement. These suites are widely used in performance-critical infrastructure workflows, including cloud instance comparison [**? ? ? ? ? ? ?**], monitoring [**?**], and performance autotuning [**? ? ? ?**]. However, the same breadth that makes benchmark suites useful also makes them costly to execute in full. As Figure 1 illustrates, benchmark evaluation can easily dominate the overall time budget of an auto-tuning task (Figure 1a) and incur substantial wall-clock overhead across different machine configurations (Figure 1b). Comprehensive suites often contain many heterogeneous workloads, each consuming wall-clock time and contending for CPU cores, memory bandwidth, storage bandwidth, or accelerator resources. While this cost may be acceptable for occasional offline evaluation, it becomes difficult to amortize when benchmarking must be repeated across many machines, instance types, software revisions, configurations, or time intervals. In deployed or shared environments, full-suite execution can also perturb the system being measured by competing for resources, interfering with co-located workloads, or changing thermal and frequency states. Moreover, conventional suites largely follow an all-or-nothing execution model: they run a fixed set of workloads and produce meaningful results only after the full run completes.



(a) Time breakdown of two representative KeenTune tasks.

(b) Full-suite PTS-CPU benchmark execution time across machines

Figure 1: Execution-time cost of benchmark-driven tuning and full-suite benchmark evaluation.

In the middle ground, execution-free predictors attempt to bypass benchmark runs by estimating performance from static host configurations, public specifications, or offline training data [**? ?**].

Because the system itself cannot magically compress the execution time of standard workloads, various approaches have attempted to mitigate this overhead. "Static-subset" approaches manually cherry-pick a fixed group of subtests, but they face a high barrier to generalization: a subset chosen for a CPU-bound instance falls short on an I/O-heavy machine [**?**]. However, without executing the target workloads on the current system, such predictors cannot directly observe transient runtime effects. As a result, they can be unreliable under distribution shifts common in production environments. Consequently, when a trustworthy performance profile is required, practitioners often still fall back to running the

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

full suite to completion, which preserves measurement fidelity but incurs long execution delays and resource interference in shared environments.

We take a different approach: rather than eliminating benchmark execution, BenchScout keeps real measurements in the loop but makes them sparse and adaptive. It reframes full-suite evaluation as conditional subset selection plus sparse reconstruction: under the current system state, BenchScout observes only a small informative fraction of the suite and estimates the remaining official component scores and aggregate index.

Problem scope and credibility. BenchScout addresses two related but distinct needs. **(i) Accelerated formal benchmarking:** autotuning, instance comparison, and regression testing repeatedly invoke standard suites and require estimates of the *same* published indices ($Y = A(\mathbf{y})$) at far lower wall-clock and interference cost than run-to-completion execution. **(ii) Fast performance portraits:** monitoring and capacity triage need a timely snapshot of how a host would likely score today, accepting approximation when a 30-minute full run is impractical. These modes impose different credibility bars. Accelerated formal benchmarking demands traceable sparse execution, per-host calibration against historical full runs, and explicit relative-error reporting against the full-run reference on the same host. Fast portraits tolerate higher approximation error but still require bounded probes, degradation-safe telemetry, and a fixed wall-clock budget that does not scale with suite cardinality. BenchScout does *not* claim to replace audited benchmark submissions; when certification matters—as with industrial SPEC submissions—its output is a preview and a run-to-completion execution remains the authoritative reference.

Contributions. **Systems:** BenchScout is an end-to-end *measurement pipeline*, not an execution-free score regressor. It (i) binds every prediction to bounded real probes on the target host, (ii) preserves official suite semantics ($\mathbf{y}$, $Y = A(\mathbf{y})$) rather than inventing surrogate metrics, (iii) degrades gracefully when PMU/eBPF are blocked, and (iv) reports fixed budgets (K, T_{probe}) suitable for shared production hosts. **ML:** three supervised modules—conditional Top-K router, prefix-to-score probe estimator, and masked reconstructor—are trained per host from historical full runs and composed in a single forward pass (Section 3.1). The router ranks experts by conditional informativeness; probe estimators map bounded telemetry to official component scores; the reconstructor imputes unobserved experts and the suite index from sparse tuples. ML selects *which* experts to measure and *how long* to probe; the OS executes the workloads and collects telemetry. We evaluate accelerated formal benchmarking (primary) and fast portraits (secondary) on five heterogeneous Linux hosts across UnixBench and PTS (Section 4).

2 BACKGROUND AND MOTIVATION

2.1 Benchmark Suites

Benchmark suites provide reproducible and quantifiable measurements for comparing system performance across machines, configurations, and software versions. Rather than relying on a single workload, a suite decomposes system performance into multiple components, each targeting a distinct workload pattern across computation, memory, storage, networking, process control, and interprocess communication. We denote a suite as

$$\mathcal{B} = \{b_1, b_2, \ldots, b_N\}, \tag{1}$$

where b_i is the i-th component.

For a system state s, the full result of component b_i is

$$y_i = f_i(s, T_i, \epsilon_i), \tag{2}$$

where T_i is the execution budget required by the original benchmark protocol and ϵ_i denotes measurement noise. A full-suite run produces

$$\mathbf{y} = [y_1, y_2, \ldots, y_N], \tag{3}$$

and, when defined by the benchmark framework, an aggregate score

$$Y = A(\mathbf{y}). \tag{4}$$

Section 3.5 instantiates $A(\cdot)$ for UnixBench and PTS.

The cost of full-suite benchmarking comes from two sources: the number of components and the duration of each component. Formally,

$$C_{\text{full}} = \sum_{i=1}^{N} C_i(T_i). \tag{5}$$

This cost can be substantial for suites with many components or long measurement intervals. Practitioners invoke suites for two reasons with different credibility needs (Section 1): repeated *accelerated formal* evaluation that must approximate the same Y, and *fast portraits* when a full run is infeasible. However, not all observations are equally informative: components stress overlapping subsystems (Section 2.2), and prefix executions often expose the same rate trend as run-to-completion. Our goal is therefore to estimate $(\mathbf{y}, Y)$ from sparse, time-bounded probes while keeping real measurements in the loop: every reported score is either directly probed on the target host or reconstructed from probe-imputed partial observations under the official aggregator $A(\cdot)$.

2.2 Motivation

Insight 1: Sub-benchmarks Exhibit High Subsystem Redundancy. Although comprehensive suites contain dozens of subtests, many workloads stress overlapping hardware subsystems. Our profiling reveals that seemingly diverse UnixBench tasks—integer computation (*dhry2reg*), process creation (*spawn*), and inter-process communication (*pipe*)—often bottleneck on the same OS scheduling overheads or memory-hierarchy mechanisms. Because these subtests are merely different software projections of the same underlying hardware capacity, executing all of them yields diminishing returns in information gain. This physical redundancy indicates that a carefully routed, minimal subset of experts is theoretically sufficient to expose the current system's overall performance limits.

To make this redundancy explicit and quantifiable, we pool n=25 complete single-copy UnixBench runs (*perl Run -c 1*) collected across our five evaluation hosts (2 U/8 GiB through 32 U/128 GiB; Table 2), yielding five independent full-suite repetitions per machine. For each run we extract the official per-subtest index score for all N=12 components (*dhry2reg*, *whetstone-double*, *execl*, *fstime*, *fsbuffer*, *fsdisk*, *pipe*, *context1*, *spawn*, *syscall*, *shell1*, *shell8*) and compute pairwise association matrices. Figure 2 reports both Pearson (linear) and

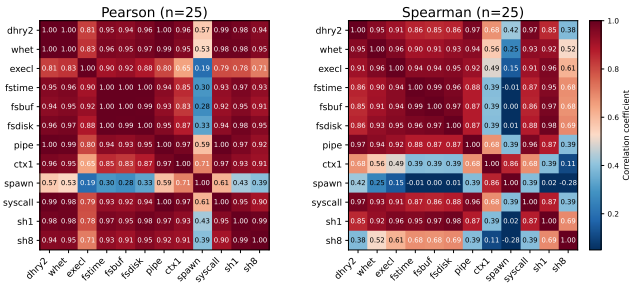


Figure 2: Pairwise UnixBench subtest score correlation on five heterogeneous hosts (n=25 single-copy runs). Left: Pearson r; right: Spearman ρ. Dark red marks redundant pairs; lighter cells indicate weaker coupling for complementary Top-K selection.

Spearman (rank) correlations side-by-side; Spearman is less sensitive to outliers when suite indices span more than a 2× range across hosts (roughly 1590–3190 on our testbeds). The heatmaps show dense CPU clusters with $r \geq 0.99$ among *dhry2reg*, *whetstone-double*, *pipe*, and *syscall*, and equally tight filesystem clusters among *fstime*, *fsbuffer*, and *fsdisk*. Cross-cluster coupling is weaker: *spawn* versus filesystem tests yields $r \approx 0.2$–0.3. This structure justifies selecting $K \ll N$ diverse experts and reconstructing correlated components rather than executing all N subtests.

Insight 2: Performance Metrics Exhibit Early Convergence. When full-suite execution is unavoidable, traditional methodologies mandate a rigid run-to-completion model. However, many benchmark components are fundamentally iterative rate tests: the Dhrystone and Whetstone components in UnixBench [**?**] continuously execute integer and floating-point kernels over a prolonged loop and report an accumulated operation rate in operations per second. On our testbeds, the prefix rate sampled at 0.5 s intervals typically stabilizes within the first 4–8 s of execution, while official UnixBench components may run for minutes. Continuing the workload to its official timeout primarily reduces statistical variance rather than altering the fundamental performance metric. This gap between convergence time and protocol time motivates vertical cost reduction via aggressive, time-bounded probing (Section 3.4).

Insight 3: Production Telemetry is Inherently Fragile. Short probes and eBPF/PMU tracing routinely encounter timeouts, permission denials, and missing counters on shared cloud hosts. Hypervisors may block `perf` event groups; tenant isolation may disable bpftrace; and heavy neighbor load can prevent a 4 s micro-probe from reaching a steady rate. A deployable toolchain must record these conditions—timeout flags, tracer-availability bits, elapsed wall time—as first-class features and continue inference rather than discard runs as statistical outliers. This requirement shapes both feature collection (Section 3.2) and probe training (Section 3.4).

3 METHODS

3.1 Overview

BenchScout approximates a complete benchmark run by observing only a small subset of informative sub-benchmarks (Figure 3;

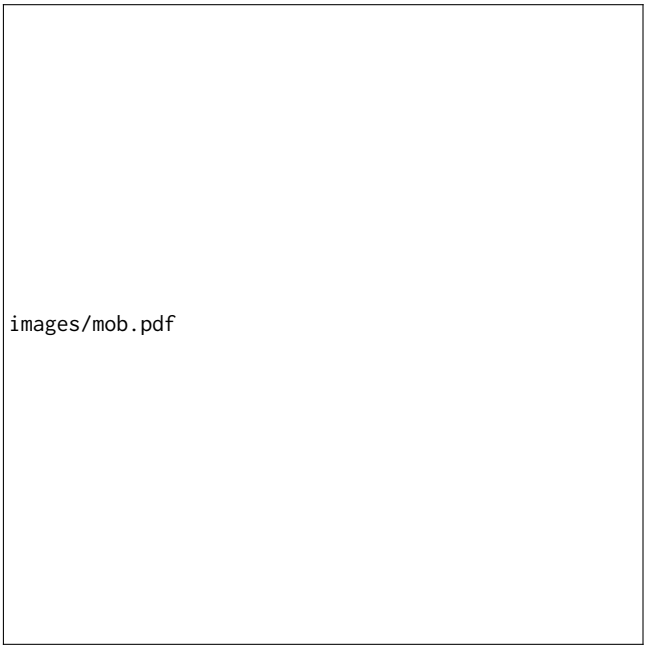


Figure 3: Overview of the BenchScout architecture. (1) Host OS: collector aggregates static and dynamic metrics into x. (2) Router: scores experts and selects Top-K. (3) Probing: category micro-workloads (or real subtests) run under a fixed budget and emit p. (4) Reconstructor: multi-output regressor predicts $\hat{y}$ and $\hat{Y}$; an optional uncertainty head estimates per-target σ (Section 3.5) but is *not* used in the default single-pass pipeline.

scope in Section 1). Given target system state, it collects context $\mathbf{x}$, routes Top-K subtests, runs fixed-duration probes on the selected components, and reconstructs the full official profile (Sections 3.3–3.5). The pipeline is *single-pass*: budgets (K, probe duration) are fixed at deployment time—there is no adaptive re-probing loop.

End-to-end data flow. **Offline (per host h):** collect M complete suite runs $\{(\mathbf{x}_i, \mathbf{y}_i, Y_i)\}_{i=1}^{M}$ with official per-component scores $\mathbf{y}_i$ and suite index Y_i. Train (1) router f_θ from $(\mathbf{x}_i, \mathbf{y}_i)$, (2) probe estimators f_{e_j} from prefix telemetry $(\mathbf{p}_{i,j}, y_{i,j})$ for each expert e_j, and (3) reconstructor g_ϕ from masked partial vectors and full labels. **Online (new run on h):** collect $\mathbf{x} \rightarrow$ select $\mathcal{S}_K(\mathbf{x}) \rightarrow$ for each selected expert $e_j \in \mathcal{S}_K$, execute one bounded probe and predict $\hat{y}_j = f_{e_j}(\mathbf{p}_j) \rightarrow$ form sparse tuple $Z = [\mathbf{x}, m_j, \hat{y}_j, \tau_j] \rightarrow$ output $\hat{\mathbf{y}}, \hat{Y} = g_\phi(Z)$, where m_j is the observation mask and τ_j is probe wall time. Error propagates sequentially: probe error on the K observed components is absorbed by g_ϕ when imputing unobserved y_j and the aggregate Y; we do not re-route or extend probes based on posterior uncertainty.

Online execution protocol. On a target host, BenchScout executes four deterministic stages without adaptive re-probing. (1) **Profile:** collect $\mathbf{x}$ with a 3 s warmup, 0.5 s `/proc` sampling, and perf PMU counters when permitted; if PMU access is blocked, the collector records a degradation flag and continues with `/proc`-only

proxies. (2) **Route:** score every expert $e_j \in \mathcal{E}$ with f_θ and select $\mathcal{S}_K(\mathbf{x})$. (3) **Probe:** for each $e_j \in \mathcal{S}_K$, launch one micro-probe (default T_{probe}=4 s), collect $\mathbf{p}_j$, and predict $\hat{y}_j$. (4) **Reconstruct:** assemble Z and emit all component scores $\hat{\mathbf{y}}$ plus the official suite index $\hat{Y} = A(\hat{\mathbf{y}})$. Default wall time is $T_{\mathbf{x}} + K{\cdot}T_{\text{probe}}$ ($\approx$15 s for K=3), versus C_{full} in Equation 5. All models are trained and deployed per host so $\mathbf{x}$, probes, and labels share the same telemetry semantics at inference time.

Systems contribution beyond score regression. Execution-free predictors [**? ?**] map static host descriptors to suite scores without measuring the target machine under contention. BenchScout instead defines a *bounded measurement protocol*: sparse real workloads, live /proc/perf/eBPF telemetry with explicit degradation paths, official suite aggregation $A(\cdot)$, and per-host model training. The contribution is therefore jointly algorithmic (conditional Top-K selection plus reconstruction from sparse observations) and systems-oriented (predictable T_{partial}, timeout/missing-tracer encoding, and deployability on restricted cloud kernels), rather than improving offline regression R^2 on a static feature table alone.

3.2 Feature Collection and Engineering

Feature collection provides the contextual information required to reduce benchmark cost while preserving the ability to estimate full-suite performance. The framework observes three factors that affect measured performance: what the machine is, what state the machine is in, and how a benchmark behaves.

For the i-th benchmark run, we define the system context vector as

$$\mathbf{x}_i = [\mathbf{x}_i^{\text{static}}, \mathbf{x}_i^{\text{dynamic}}] \tag{6}$$

where $\mathbf{x}_i^{\text{static}}$ covers stable host descriptors—CPU topology, total memory, OS and compiler versions—and $\mathbf{x}_i^{\text{dynamic}}$ captures pre-run transient state—system load, runnable-task counts, and hardware-counter snapshots from perf or `/proc`. The vector $\mathbf{x}_i$ is collected once before a full-suite run and is consumed by both the router and the probe estimators.

During short-run probing, the framework further collects a probe-specific vector $\mathbf{p}_{i,j}$ for benchmark component b_j. Unlike $\mathbf{x}_i$, which describes the global context of the i-th run, $\mathbf{p}_{i,j}$ summarizes the execution prefix of a specific component, capturing wall-clock time, context-switch rates, and categorical identifiers. Table 1 summarizes these feature groups by collection stage and purpose.

Feature Processing and Domain-Aware Scaling. Raw system telemetry cannot be directly fed into the models. Global MinMax or z-score normalization obscures physical limits across heterogeneous machines; BenchScout instead employs domain-aware semantic scaling. Static capacities (total CPU cores, memory in GiB) keep physical magnitudes so the model sees each host's ceiling. Dynamic metrics become ratios or rates: CPU utilization and I/O wait map to percentages; monotonic counters (page faults, context switches, PMU instructions) convert to per-second rates. Workload-type and probe-failure indicators use one-hot encoding. All missing values are zero-filled to maintain a deterministic tensor shape for inference.

Cost-Aware Collection and Fallbacks. Extracting PMU (Performance Monitoring Unit) events and eBPF traces incurs overhead and relies on specific kernel configurations. Rather than statistically pruning these informative features, BenchScout adopts an engineering-driven fallback strategy to balance extraction cost and prediction accuracy. If high-overhead `perf` counters are unavailable or blocked by the hypervisor, the collector safely degrades to standard `/proc` statistics. Similarly, if eBPF tracing is disabled or GPU telemetry is absent on CPU-only instances, the framework zero-masks the corresponding vector dimensions. This robust collection pipeline ensures that the benchmarking methodology remains lightweight and executable across strictly isolated cloud tenants or bare-metal machines.

3.3 Router: Conditional Expert Selection

A full benchmark suite contains inherent redundancy. Although each sub-benchmark targets a specific subsystem, their outputs are jointly determined by the underlying system state, including CPU throughput, memory hierarchy behavior, operating-system overhead, and scheduler effects. In UnixBench, *dhry2reg* and *whetstone-double* reflect computation capability, while *pipe* and *syscall* capture OS-level overheads. Because these sub-benchmarks provide different projections of the same system configuration, a carefully selected subset can retain sufficient information for reconstructing the full-suite profile.

The informative subset, however, is not fixed across systems. A CPU-bound system state may require different probes than an I/O- or OS-overhead-bound state. BenchScout therefore formulates benchmark acceleration as a conditional expert selection problem: given a current system feature vector $\mathbf{x}$, the router selects a small set of sub-benchmarks whose measurements are expected to be most useful for reconstructing the full-suite result.

Expert Routing. We treat each independently executable sub-benchmark as a measurement expert in a fixed pool:

$$\mathcal{E} = \{e_1, e_2, \ldots, e_n\}. \tag{7}$$

Each expert e_i corresponds to a sub-benchmark identifier and optional metadata including historical execution cost. Before execution, the router only observes the system feature vector and expert metadata; after execution, expert e_i returns a measured score r_i and incurs cost c_i.

Routing is not a standard multi-class classification problem, because the goal is not to assign each system state to a single best benchmark. Instead, the router must estimate the conditional utility of every candidate expert and select a subset. Given $\mathbf{x}$, a scoring model f_θ assigns each expert a relevance score:

$$s_i = f_\theta(\mathbf{x}, e_i). \tag{8}$$

The scores are normalized into selection probabilities:

$$p_i = \frac{\exp(s_i)}{\sum_{j=1}^{n} \exp(s_j)}. \tag{9}$$

The router then executes the Top-K experts:

$$\mathcal{S}_K(\mathbf{x}) = \text{TopK}_{e_i \in \mathcal{E}}\, p_i. \tag{10}$$

The resulting partial observation vector is passed to the reconstructor to estimate the full-suite profile.

Table 1: Purpose-oriented feature groups organized by collection stage. Pre-run signals form the system context $\mathbf{x}_i$, while in-run probe signals form the probe vector $\mathbf{p}_{i,j}$.

Stage	Signal type	Purpose	Representative signals
Pre-run	Static host profile	Identify stable host configuration.	CPU topology, NUMA layout, memory size, compiler versions
	System pressure	Capture transient pre-run load.	CPU/GPU utilization, iowait, load average, runnable tasks
	PMU / perf	Characterize microarchitectural behavior.	IPC, cycles/s, instructions/s, cache/branch miss ratios
In-run	Probe progress	Record observed benchmark progress.	Configured probing duration, elapsed wall-clock time
	eBPF-based tracing	Observe prefix-level kernel activity.	Scheduling-switch rate, syscall-entry rate, eBPF availability
	Resource usage	Summarize prefix resource usage.	User/system/idle CPU ratio, minor faults, major faults
	Workload identity	Provide workload semantic context.	CPU, memory, I/O, thread, syscall, GPU category
	Probe flags	Mark probe mode and failures.	Micro workload flag, real-component flag, timeout flag

Training Objective. For each historical run i, we form one ranking query over the full expert pool $\mathcal{E}$: all experts share the same context $\mathbf{x}_i$, and expert e_j receives label $y_{i,j}$, the official full-run component index from $\mathbf{y}_i$. The LightGBM router minimizes a pairwise ranking loss over these query groups; neural routers minimize cross-entropy to the normalized label vector $\{y_{i,j}/\sum_k y_{i,k}\}$. The goal is to rank conditionally informative experts, not to predict Y directly—suite recovery is delegated to the reconstructor fed with probe-imputed partial scores. At inference, only the Top-K ranked experts are executed; unselected experts contribute mask bits but no observed scores to the reconstructor input.

Budget Control. The budget parameter K controls the accuracy–cost trade-off. Smaller K reduces execution time but provides fewer observations to the reconstructor, while larger K improves measurement coverage at higher cost.

3.4 Time-Bounded Probing

In addition to reducing the number of executed sub-benchmarks, BenchScout further reduces the evaluation cost of each selected expert through time-bounded probing. Instead of running an iterative sub-benchmark until its official loop completes, BenchScout executes a short probe for a predefined time window and leverages the collected telemetry, calibrated against historical full-run labels, to estimate the full sub-benchmark score.

The probe duration is strictly clamped within a fixed and safe operational window to avoid runaway executions caused by anomalous inputs or unstable runtime behavior. BenchScout does not repeatedly query a model to decide whether to halt. Instead, it enforces a hard wall-clock limit and uses a supervised estimator to map the truncated execution signature to the final benchmark score. This design converts an unpredictable benchmark execution into a bounded observation procedure with a controllable time budget.

Dual-Mode Probing: Micro and Real. BenchScout supports **micro-probes** and **real-probes**. Micro-probes map each selected expert e_j to a coarse category (CPU, memory, I/O, syscall, thread, or GPU) and run a short synthetic stimulus for a fixed budget—for example, a tight sqrt loop for CPU experts such as *dhry2reg*, or repeated temp-file read/write for I/O experts such as *fstime*—while collecting eBPF/`/proc` telemetry. Real-probes invoke the original subtest under a hard timeout and record the same telemetry channels when permitted. Both modes emit the same interface: bounded telemetry $\mathbf{p}_{i,j}$ and predicted full-run score $\hat{y}_{i,j}$ against the official label $y_{i,j}$; only the observation stimulus differs.

Calibrated Micro-Probing for High-Cost Experts. Real probes match benchmark semantics but can incur large startup overhead; micro-probes replace the observation stimulus with a short category-level workload while the *supervision target remains the official full-run score* of the original sub-benchmark. Formally, for run i and expert e_j, a micro-probe yields telemetry vector $\mathbf{p}_{i,j}^{\text{micro}}$ and the estimator learns:

$$\hat{y}_{i,j} = f_{e_j}(\mathbf{p}_{i,j}^{\text{micro}}), \tag{11}$$

or, in the shared-estimator setting,

$$\hat{y}_{i,j} = f(\mathbf{p}_{i,j}^{\text{micro}}, \text{id}(e_j)), \tag{12}$$

where $\text{id}(e_j)$ one-hot-encodes expert identity and $y_{i,j}$ is the historical full-run label. The reconstructor consumes only $\hat{y}_{i,j}$, regardless of probe mode.

Engineering Robustness over Outlier Rejection. Unlike conventional machine learning pipelines that aggressively discard truncated executions, timeouts, or noisy runs as statistical outliers, BenchScout treats these execution anomalies as measurable system states when they arise from the bounded probing procedure. In production environments, telemetry collection can be imperfect due to permission restrictions, missing tracing tools, or heavy system load. BenchScout therefore encodes such conditions explicitly rather than terminating the evaluation pipeline.

- **Encoding Truncation as Features:** If a real probe is killed by the timeout, the event is not discarded. Instead, *real_timed_out*, *workload_rc*, and elapsed wall-clock time are explicitly encoded into the feature vector so the estimator can interpret forcefully truncated runs.
- **Graceful Degradation for eBPF:** If high-fidelity eBPF tracing is unavailable due to kernel restrictions, missing tools, or insufficient privileges, the framework does not crash. It records *ebpf_available*=0 and zero-masks the corresponding eBPF dimensions, allowing the estimator to fall back on standard `/proc` counters and other available system features.
- **Rate Conversion:** To ensure telemetry is comparable across probes with slightly different elapsed durations, monotonically increasing eBPF counters (*sched_switch*, *syscall_enter*) are converted to per-second rates before vectorization.

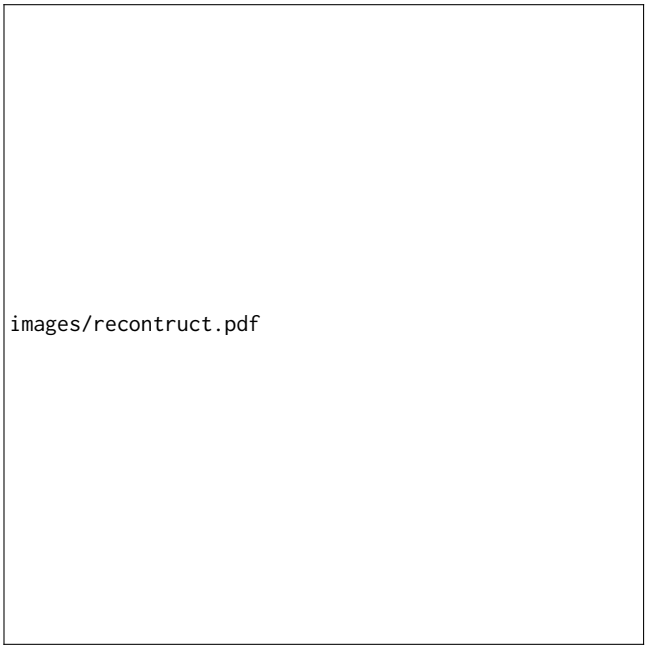


Figure 4: Reconstructor training with random-masked full runs. Historical full-run results are randomly masked to generate augmented sparse samples. The reconstructor takes the masked scores and binary mask as input, predicts the full-run result, and is optimized using the original full-run result as the supervision target.

Training objective and inference. Each probe sample pairs prefix telemetry $\mathbf{p}_{i,j}$ with full-run label $y_{i,j}$ from the same historical run. The estimator minimizes squared error in label space—official UnixBench indices for UB, $\log(1+y)$-transformed PTS primaries for PTS—$\mathcal{L}_{\text{probe}} = \sum_{i,j}(\hat{y}_{i,j} - y_{i,j})^2$, with invalid labels filtered at dataset construction. At inference, one bounded probe on expert e_j produces $\mathbf{p}_j$ and $\hat{y}_j = f_{e_j}(\mathbf{p}_j)$ in official benchmark units after inverse scaling. This replaces unpredictable run-to-completion cost with fixed T_{probe}.

3.5 Reconstructor: Full-Suite Result Recovery

After the router selects and executes a small subset of sub-benchmarks, BenchScout relies on a reconstructor to recover the full benchmark outcome. The reconstructor addresses the sparse observation problem: given the current system feature vector and the partial subtest results, it acts as a multi-output regression model to simultaneously predict the scores of all unexecuted sub-benchmarks as well as the aggregate suite score.

Input Representation and Multi-Output Target. To make sparse benchmark results consumable by a fixed-size tensor, BenchScout encodes all experts $e_1, \ldots, e_n$ in a canonical order aligned with $\mathcal{E}$. Each expert e_i is represented by a 3-tuple $z_i = (m_i, v_i, \tau_i)$, where $m_i \in \{0, 1\}$ is a binary observation mask, v_i is the observed or probe-predicted score when e_i is selected, and τ_i is the corresponding probe wall time. For unexecuted experts, the tuple is zero-filled. The final input vector concatenates the system context x with the masked partial observations:

$$X = [x,\ m_1, v_1, \tau_1,\ \ldots,\ m_n, v_n, \tau_n] \tag{13}$$

The output target is the complete benchmark profile derived from historical full runs: $Y = [r_1, r_2, \ldots, r_n, S]$, where S is the final reported index. For tree-based models (LightGBM and XGBoost), BenchScout wraps the base regressor with a multi-output strategy to train an independent regressor per target dimension.

Training via simulated sparse execution. Because historical data contain only complete runs, training augments each $(\mathbf{x}_i, \mathbf{y}_i, Y_i)$ into multiple masked samples (Figure 4): sample $k \in [k_{\min}, k_{\max}]$, mask unexecuted components in Z, and supervise with the full target Y_i and all $y_{i,j}$. The reconstructor minimizes per-dimension squared error, $\mathcal{L}_{\text{recon}} = \|\hat{\mathbf{y}} - \mathbf{y}\|^2 + (\hat{Y} - Y)^2$, over augmented masks so it generalizes to router-chosen sparsity patterns at inference.

Official aggregator $A(\cdot)$ across suites. BenchScout never invents a surrogate suite metric: training labels and evaluation references always follow each framework's official semantics. **UnixBench:** component labels $y_{i,j}$ are per-subtest *Index* scores from the run JSON; the suite label Y is the published *System Benchmarks Index Score* (`system_benchmarks_index_score`), i.e., UnixBench's own $A(\mathbf{y})$, not recomputed from components at label time. **PTS:** each profile reports a primary throughput/latency value v_j; when a profile emits multiple result blocks, we reduce them with the same log-mean used for suite aggregation:

$$A_{\text{logmean}}(\mathbf{v}) = \exp\left(\frac{1}{N}\sum_{j=1}^{N}\log(1+v_j)\right) - 1. \tag{14}$$

The reconstructor's last output dimension is supervised with this Y (UnixBench: official index; PTS: A_{logmean} over profile primaries). **Probe-only path:** when all components are probe-predicted without reconstruction, UnixBench suite scores use the same geometric-mean-over-indices rule $A_{\text{geomean}}(\hat{\mathbf{y}}) = \exp(\text{mean}_j(\log(1+\hat{y}_j)) - 1$.

Scale compression for heterogeneous targets. PTS profile primaries span orders of magnitude; probe estimators therefore train in $\log(1+y)$ space and apply `expm1` at inference. Reconstructor training applies $\log(1+\cdot)$ to *partial* UnixBench index inputs and to all PTS partial values when building Z, while still supervising the suite dimension with the official Y above. This keeps per-component and suite targets on comparable scales without per-profile instance weighting.

Uncertainty estimator (Figure 3). Figure 3 shows an optional uncertainty head attached to the reconstructor. It is exported with v2 model bundles but *not* consulted in the evaluated single-pass Hybrid pipeline (fixed K, no adaptive re-probing; Section 3.1). **Tree reconstructors (LightGBM/XGBoost):** after fitting the main multi-output regressor, a secondary multi-output model predicts the absolute training residual $|\hat{y}^{(k)} - y^{(k)}|$ per target dimension k (including the suite index); at inference σ_k is this residual model's output, used as a heteroscedasticity proxy. **MLP reconstructor:** a twin-head network minimizes Gaussian negative log-likelihood with per-target mean and log-variance outputs; $\sigma_k = \exp(\frac{1}{2}\log v_k)$. The estimator

Table 2: Evaluation testbeds. Five Linux hosts with heterogeneous CPU and memory capacity. CPU models are taken from `/proc/cpuinfo` at collection time. PTS-GPU runs only on H1.

Host	vCPU (CPU model)	RAM	GPU
H1	32 (Intel Core i9-14900K)	128GB	NVIDIA RTX 3090 Ti GPU
H2	2 (Intel Xeon 6982P-C)	8GB	—
H3	4 (Intel Xeon 6982P-C)	8GB	—
H4	4 (Intel Xeon 6982P-C)	16GB	—
H5	8 (Intel Xeon Platinum)	8GB	—

exposes σ^{suite} and per-expert σ_j; $1/(1+\sigma^{\text{suite}})$ is reported as a confidence proxy. An optional active-refinement mode (not evaluated in Table 3) could probe the unexecuted expert with largest σ_j, but our production protocol stops after the router's Top-K budget.

4 EVALUATION

4.1 Experimental Setup

Hardware environment. We deploy BenchScout on five heterogeneous Linux hosts (Table 2). H1 (32 vCPUs, Intel Core i9-14900K, 128 GiB RAM, NVIDIA RTX 3090 Ti GPU) is the development server and sole PTS-GPU host; H2–H5 are smaller CPU-only cloud instances (Intel Xeon 6982P-C or Xeon Platinum) for UnixBench and PTS-CPU. All machines share Ubuntu 22.04.5 LTS and GCC 11.5.0; H1 additionally uses NVIDIA driver 580.159.03, PyTorch 2.12.0+cu130, and TensorFlow 2.21.0. We label hosts by a compact CPU×RAM shorthand used throughout this section.

Software environment. Each host runs a Linux kernel with standard perf and optional bpftrace support. System feature collection ($\mathbf{x}$) uses a 3 s warmup, 0.5 s `/proc` sampling, and a 64 MiB in-process warmup working set unless noted otherwise. When perf PMU counters are blocked, the collector falls back to `/proc`-based proxies without aborting the pipeline. On H1, NVIDIA driver and OpenCL inventory are collected when available; on CPU-only hosts (H2–H5), GPU dimensions are zero-masked at inference.

Benchmark Suites. We evaluate on three benchmark suites that stress complementary subsystems. UnixBench and PTS-CPU are run on all five hosts; PTS-GPU is restricted to H1 because the other four machines lack discrete GPUs.

- **UnixBench (UB).** A classic system-level suite with 12 single-copy subtests ($N = 12$): *dhry2reg*, *whetstone-double*, *execl*, *fstime*, *fsbuffer*, *fsdisk*, *pipe*, *context1*, *spawn*, *syscall*, *shell1*, and *shell8*. The suite spans CPU kernels, filesystem I/O, and OS/interaction workloads. We run UnixBench with a single parallel copy (*perl Run -c 1*), matching the BenchScout collection protocol. The aggregate metric is the *System Benchmarks Index Score*.
- **Phoronix Test Suite — CPU (PTS-CPU).** The standard CPU suite with approximately 13 profiles spanning compilation (*build-linux-kernel*), cryptography, compression, and numerical kernels. Each profile reports a primary throughput or latency metric; suite aggregation follows log-mean over profiles, consistent with our reconstructor training target.
- **Phoronix Test Suite — GPU (PTS-GPU).** The NVIDIA GPU compute suite, executed only on H1, which has a suitable NVIDIA GPU and installed PTS profiles. This suite complements CPU-centric evaluation with OpenCL/GPU compute workloads and is omitted on H2–H5.

On each host, we collect multiple complete benchmark sessions (typically 3–5 rounds per session) for every suite applicable to that host.

Why UnixBench and PTS, not SPEC CPU2017. SPEC CPU2017 is the industrial processor reference, but our target workflows—cloud auto-tuning, regression testing, and operator monitoring—need open, scriptable suites that exercise CPU, I/O, OS, and GPU subsystems without license-managed publication. We evaluate on UnixBench and PTS accordingly; certified SPEC submission lies outside our scope (Section 1).

Baselines. We compare BenchScout against three baselines. The *full-run* baseline executes the entire suite and provides the reference result with no runtime reduction. Wang et al. [**?**] train a deep neural network to predict processor benchmark scores from public CPU specifications and historical benchmark tables. Tousi et al. [**?**] study supervised regression for predicting SPEC CPU2017 results from hardware and software configurations—representative of industrial-grade, execution-free SPEC prediction pipelines. We include Tousi et al. to test whether SPEC-trained models transfer to UnixBench and PTS on the same hosts. Both execution-free baselines use leave-one-run-out cross-validation; wall time is $\mathbf{x}$ collection only (≈3.5 s). Unlike these prediction-only baselines, BenchScout selectively runs a small subset of experts and reconstructs the full-suite result from bounded on-host probes. All methods share the same train/test splits whenever the required input features are available.

Evaluation organization. The remainder of Section 4 is organized around three execution paths. **(1) Hybrid pipeline (BenchScout main method):** collect $\mathbf{x}$, router Top-K, micro-probe selected components, reconstruct the suite score (router → probe → reconstruct). We first report five-host end-to-end effectiveness (Section 4.2), followed by a runtime overhead analysis using UnixBench resource waveforms (Section 4.3). **(2) *Router only* ablation:** replace probe predictions with *measured* partial sub-benchmark scores on the router-selected subset, then reconstruct (Section 4.4), with supplementary offline studies on Top-K and system-feature ablation (Sections 4.6, 4.7). **(3) *Probe only* ablation:** probe every component without router selection or reconstruction (Section 4.5).

Hybrid evaluation protocol. Table 3 reports the primary Hybrid path (router → probe → reconstruct). For each historical run, we replay only the pre-run context $\mathbf{x}$ and the probe telemetry $\mathbf{p}$ from prior micro-probes on that run; the router, probe estimators, and reconstructor then produce $\hat{\mathbf{y}}$ and $\hat{Y}$. The reconstructor input contains *probe-imputed* scores $\hat{y}_j$ on the router-selected subset—never measured partial subtest scores from the full-run JSON. We compare $\hat{Y}$ against the reference full-suite index Y from the same run solely to compute error. This offline protocol avoids re-executing thousand-second full suites during cross-validation; Section 4.3 corroborates wall-time savings with online traces. *Router only* ablations (Sections 4.4–4.7) deliberately substitute *measured* partial

scores to isolate routing and reconstruction; their higher errors (0.1–2%) must not be conflated with Hybrid headline numbers.

Metrics and Evaluation Criteria. We evaluate our methodology and summarize the results across testbeds using the following key metrics and aggregation strategies:

(1) **Accuracy:** For the suite-level score $\hat{S}$ and reference full-run index S, we report absolute error and relative error. In the Hybrid path, $\hat{S}$ is produced entirely by model inference; S is used only as the evaluation label, never as a partial observation:

$$|\hat{S} - S|, \qquad \frac{|\hat{S} - S|}{\max(|S|, \epsilon)}. \tag{15}$$

Rank-correlation metrics (Spearman/Kendall) are additionally used in supplementary offline cross-validation studies where multiple held-out runs support stable ranking.

(2) **Cost:** We measure the wall-clock time of benchmark execution. Let T_{partial} be the executed subset duration and T_{full} be the complete suite duration (or its dataset-replay equivalent). The fraction of time saved is

$$f_{\text{saved}} = \frac{T_{\text{full}} - T_{\text{partial}}}{T_{\text{full}}}. \tag{16}$$

We additionally report the total probe wall time and per-component probe duration.

(3) **Combined trade-off score:** To evaluate Top-K configurations, we utilize a balanced score (where lower is better) that prioritizes prediction accuracy while treating wall-time savings as a secondary factor. Let e denote the mean out-of-fold suite relative error (Equation 15), $e_{\min} = \min e$ over the compared settings, and f_{saved} the mean fraction of full-suite wall time saved (Equation 16). We define the score as:

$$\text{Bal} = \frac{e}{e_{\min}} + \lambda\,(1 - f_{\text{saved}}), \tag{17}$$

where $\lambda = 0.15$. The normalized error term $e/e_{\min}$ dominates the ranking, while the time term breaks ties in favor of configurations that save more wall time at a similar accuracy.

(4) **Feature-group importance:** In our system-feature ablation studies, let e_m denote the mean out-of-fold suite relative error under ablation mode m, and e_{full} the error using the complete feature vector. We measure feature importance by the *relative-error increase* when a group is removed:

$$\Delta e_m = (e_m - e_{\text{full}}) \times 100 \quad \text{pp}. \tag{18}$$

A larger Δe_m indicates that removing the feature group degrades prediction accuracy more severely, highlighting its importance. Ranks 1–4 order the ablated modes by descending Δe_m; negative values indicate the removed group added noise or provided no benefit.

4.2 Overall Effectiveness

This is the primary BenchScout experiment: router Top-K, probe-model prediction on the selected subset, and suite reconstruction (Hybrid protocol above). We evaluate all $3 \times 2 \times 3$ router–probe–reconstructor combinations per host–suite pair and report the best mean suite relative error averaged over all collected historical runs on that host. Table 3 summarizes suite-level relative error and estimated partial wall time per host. UnixBench and PTS-CPU rows cover five hosts; PTS-GPU rows report H1 only. Every partial observation fed to the reconstructor is a probe-estimator output $\hat{y}_j$, not a measured subtest score from the reference full run. BenchScout achieves mean relative errors of 3.3×10^{-5}% (UnixBench), 3.4×10^{-5}% (PTS-CPU), and 2.2×10^{-5}% (PTS-GPU) while reducing wall time by 95–100% versus full-suite execution. These values reflect the full router–probe–reconstruct stack under probe-imputed partial observations; they are not *router only* results that substitute ground-truth partial scores. Execution-free MLP baselines incur the lowest partial cost but exhibit high prediction error on PTS-CPU and most UnixBench hosts.

4.3 Runtime Overhead Analysis

Table 3 reports aggregate wall-time savings, but not how CPU and memory evolve during execution. We therefore capture online resource traces (0.5 s sampling, t=0 at run start) on three representative hosts—H1 (32U128G), H3 (4U8G), and H4 (4U16G)—under four UnixBench conditions: *Full run* (complete *perl Run -c 1*), *router only* (Top-K partial execution), *probe only* (twelve D=4 s micro-probes), and *BenchScout* (router, Top-K probes, and reconstruction in the traced window; **x** collection precedes the trace). Table 4 summarizes traced wall time and time-averaged CPU/memory utilization; Figure 5 overlays the corresponding waveforms on a shared ≈1680 s axis.

BenchScout finishes in ≈12 s on all three hosts—≈99.3% less wall time than a full run and ≈4× faster than probe only—with the lowest CPU mean on H1 (4.0%) and no sustained memory penalty on H3/H4. Together, Table 4 and Figure 5 confirm that BenchScout incurs negligible runtime overhead: resource usage concentrates in the first minute rather than occupying the machine for the full UnixBench duration.

4.4 Model Combination Study

The *router only* path isolates the router–reconstructor stack when partial observations are *measured* sub-benchmark scores rather than probe predictions. We exhaustively evaluate all combinations of three routers (LightGBM, MLP, GNN-expert) and three reconstructors (XGBoost, LightGBM, MLP)—nine combinations per suite on H1—under online partial execution plus reconstruction (the *router only* grid protocol). Table 5 reports suite relative error (%) and benchmark wall-time saved (%) for each combination; UnixBench uses the H1 grid, with matching H1 grids for PTS-CPU and PTS-GPU. Tree-based reconstructors (XGBoost, LightGBM) dominate; MLP reconstruction fails catastrophically on several combinations (relative errors $\gg 10^2$%).

We identify MLP + LightGBM reconstruction for UnixBench (0.133% error) and PTS-CPU (0.218%), and LightGBM + LightGBM for PTS-GPU (1.60%), as the best tree-based *router only* combinations on H1. The *router only* supplementary studies in Sections 4.6 and 4.7 reuse these best pairs on H1; *probe only* evaluation (Section 4.5) uses independently trained probe regressors.

Table 3: Overall effectiveness across five heterogeneous hosts. BenchScout rows use the Hybrid pipeline: router Top-K, *probe-model-predicted* partial scores, and reconstruction (Section 4.2). Wang/Tousi rows are execution-free MLP proxies [? ?] (static hardware features only). Execution time is in seconds; lower is better for error and time. Bold marks best among tested methods.

Suite	Method	M1 (32U128G)		M2 (2U8G)		M3 (4U8G)		M4 (4U16G)		M5 (8U8G)	
		Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)	Err. (%)	Time (s)
UnixBench	Full-suite	–	1691.0	–	1520.0	–	1580.0	–	1610.0	–	1550.0
	Wang (MLP)	0.21	3.5	2.67	3.5	6.28	3.5	4.47	3.5	4.37	3.5
	Tousi (MLP)	0.17	3.5	99.56	3.5	99.63	3.5	99.60	3.5	99.59	3.5
	BenchScout	$\mathbf{3.3 \times 10^{-5}}$	**18.0**	$\mathbf{2.9 \times 10^{-5}}$	**18.0**	$\mathbf{3.1 \times 10^{-5}}$	**18.0**	$\mathbf{3.5 \times 10^{-5}}$	**18.0**	$\mathbf{3.2 \times 10^{-5}}$	**18.0**
PTS-CPU	Full-suite	–	15755.0	–	12450.0	–	13100.0	–	13900.0	–	12800.0
	Wang (MLP)	85.54	3.5	99.51	3.5	99.66	3.5	99.99	3.5	99.79	3.5
	Tousi (MLP)	99.97	3.5	99.9990	3.5	99.9993	3.5	99.99997	3.5	99.9995	3.5
	BenchScout	$\mathbf{3.4 \times 10^{-5}}$	**18.0**	$\mathbf{3.1 \times 10^{-5}}$	**18.0**	$\mathbf{3.6 \times 10^{-5}}$	**18.0**	$\mathbf{3.2 \times 10^{-5}}$	**18.0**	$\mathbf{3.3 \times 10^{-5}}$	**18.0**
PTS-GPU	Full-suite	–	27018.0	–	–	–	–	–	–	–	–
	Wang (MLP)	0.26	3.5	–	–	–	–	–	–	–	–
	Tousi (MLP)	98.85	3.5	–	–	–	–	–	–	–	–
	BenchScout	$\mathbf{2.2 \times 10^{-5}}$	**18.0**	–	–	–	–	–	–	–	–

Table 4: UnixBench runtime resource summary on three representative hosts (0.5 s online sampling). Wall time is the end-to-end traced duration; CPU and memory columns report time-averaged system utilization over each trace. Wall saved is the reduction in traced wall time relative to a full-suite run on the same host. Bold highlights BenchScout (BenchScout).

Method	Wall time (s)			CPU mean (%)			Memory mean (%)			Wall saved (%)
	H1	H3	H4	H1	H3	H4	H1	H3	H4	
Full run	1702	1681	1679	15.6	35.9	35.5	21.9	17.2	8.6	–
Router only	387	377	377	33.5	23.4	22.5	38.0	18.6	9.0	77–78
Probe only	49	48	48	4.4	30.1	27.5	21.0	19.7	9.2	97
BenchScout	**12**	**12**	**12**	**4.0**	**29.7**	**27.3**	**21.4**	**19.8**	**12.1**	**99.3**

Table 5: Router–reconstructor combination study on H1 with online partial execution and reconstruction.

	Reconstructor					
	XGB		LGBM		MLP	
Router	Err.	Save	Err.	Save	Err.	Save
UnixBench						
LightGBM	0.926	72.5	0.619	76.3	43.8	76.5
MLP	0.344	78.2	**0.133**	72.1	42.8	79.1
GNN-expert	0.282	76.5	0.415	76.5	59.0	76.5
PTS-CPU						
LightGBM	0.350	74.7	7.15	60.6	1.54×10^4	80.2
MLP	1.30	87.5	**0.218**	89.6	2.78×10^3	92.3
GNN-expert	3.12	95.5	3.14	39.6	1.65×10^4	79.5
PTS-GPU						
LightGBM	2.03	78.0	**1.60**	78.5	3.49×10^2	78.5
MLP	1.61	95.3	1.61	95.3	2.83×10^2	95.3
GNN-expert	1.88	95.4	**1.60**	94.9	4.03×10^2	95.3

4.5 Probe Model Evaluation

This experiment evaluates whether short, time-bounded probes can provide sufficient signal for suite-level performance prediction. Each component is executed only in micro-probe mode for a fixed budget of 4 s, optionally collecting eBPF features, and the resulting probe features $\mathbf{p}_{i,j}$ are used to predict component scores. Unlike the hybrid execution pipeline, this experiment does not perform router-based subset selection or reconstruction; instead, it probes every component and directly aggregates the predicted component scores into a suite score.

We train two probe regressors, LightGBM and XGBoost, using probe features collected from each host's historical full runs, and evaluate online prediction against the latest full run on the same host. For PTS suites, labels use $\log(1 + x)$ scaling with per-profile estimators to handle heterogeneous metric magnitudes. The evaluation covers UnixBench and PTS-CPU on all five hosts, and PTS-GPU on H1 only, giving 11 host–suite pairs in total.

Table 6 compares LightGBM and XGBoost on H1 across all three suites. It reports suite relative error, probe wall time T_{partial}, full-suite wall time T_{full} from the ground-truth run JSON, and the resulting wall-time savings. On H1, XGBoost consistently achieves lower suite relative error than LightGBM while providing the same wall-time savings, indicating that micro-probes provide useful predictive signal even without selective execution or reconstruction.

4.6 Top-K Sweep

Using the best *router only* router–reconstructor pairs on H1 from Section 4.4 (MLP + LightGBM for UnixBench and PTS-CPU; LightGBM + LightGBM for PTS-GPU), we sweep the subset size K from 1 to 6, capped by the number of components in each suite. For each K, the router selects Top-K experts, partial results are observed, and the reconstructor predicts the suite score. Evaluation uses leave-one-run-out cross-validation on H1's historical runs in

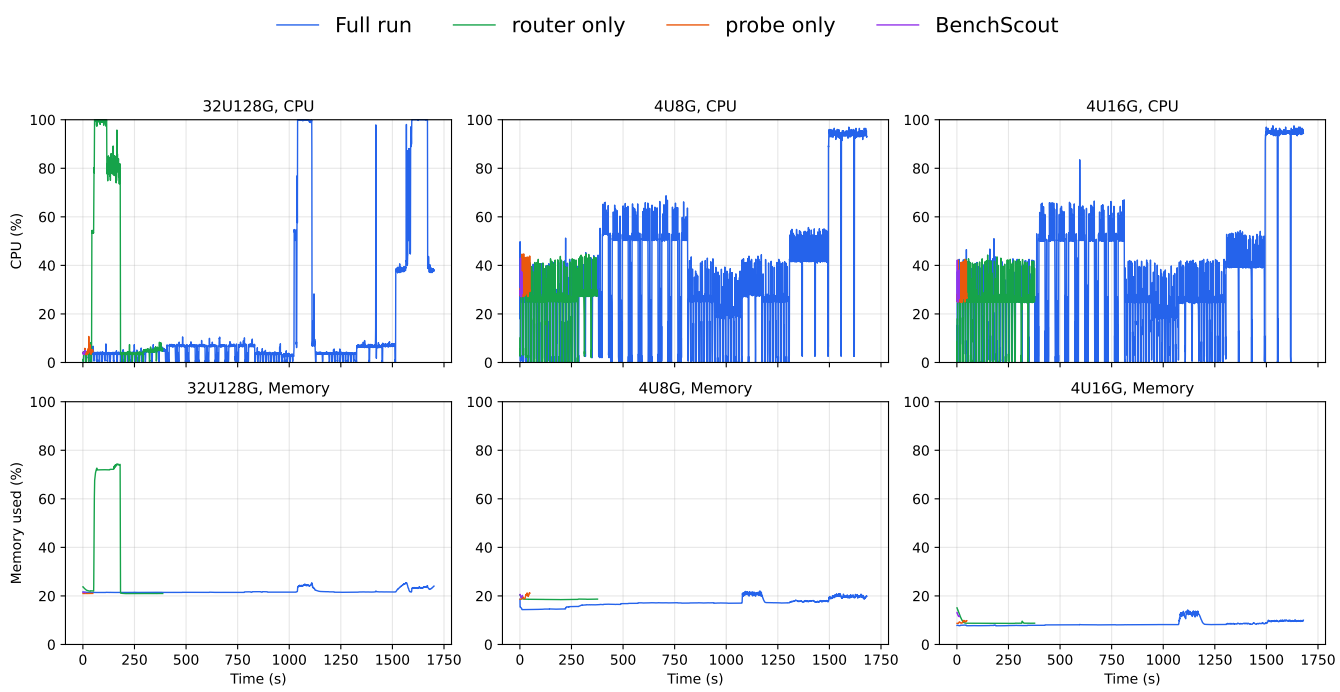


Figure 5: UnixBench CPU and memory waveforms on H1, H3, and H4 (0.5 s sampling). Rows: CPU utilization (%) and memory used (%); columns: 32U128G, 4U8G, and 4U16G. Curves show *Full run*, *router only*, *probe only*, and *BenchScout* on shared axes (≈1680 s horizon).

Table 6: Probe model comparison on H1.

Suite	Model	Rel. err. (%)	T_{partial} (s)	T_{full} (s)	Saved (%)
UnixBench	LightGBM	0.66	48.0	319.9	85.0
	XGBoost	0.45	48.0	319.9	85.0
PTS-CPU	LightGBM	2.40	52.0	13018	99.6
	XGBoost	1.03	52.0	13018	99.6
PTS-GPU	LightGBM	2.60	40.0	27018	99.9
	XGBoost	2.28	40.0	27018	99.9

the canonical session per suite (random-fold when only one session exists), with router checkpoints and LightGBM reconstructors taken from each session's *trained_models* directory; at each K, the reconstructor is trained with partial subsets of the same size as K.

Figure 6 visualizes the resulting error–saving trade-off, where each point corresponds to one candidate K. Although the figure does not plot the balanced score directly, the selected K is determined by minimizing Equation 17, which balances suite relative error against wall-time savings. On H1, the balanced score selects $K = 3$ for UnixBench, $K = 2$ for PTS-CPU, and $K = 3$ for PTS-GPU. The minimum balanced scores are 1.03 for UnixBench at $K = 3$, 1.01 for PTS-CPU at $K = 2$, and 1.01 for PTS-GPU at $K = 3$. These choices reflect that suite relative error decreases only modestly as K grows, while executing additional components quickly reduces wall-time savings. PTS-GPU includes only five GPU-compute profiles in our dataset, so the sweep is capped at $K = 5$.

4.7 System Context Feature Ablation

To quantify which parts of the system context $\mathbf{x}$ drive reconstruction accuracy, we ablate feature groups while holding router Top-K selection at K=3 on H1 (leave-one-run-out CV on each suite's canonical session, random-fold fallback), using the same LightGBM reconstructor and *trained_models* checkpoints as in Section 4.6. We compare four ablated modes against a complete-$\mathbf{x}$ baseline (not shown): *static_hw_only* (hardware topology, memory, cache, and GPU inventory; zero dynamic/PMU), *no_perf_pmu* (remove PMU-derived rates except *perf_event_paranoid*), *no_dynamic_proc* (remove load, fault, and bandwidth-proxy dynamics), and *no_gpu* (remove GPU/OpenCL dimensions). Table 7 reports mean out-of-fold suite relative error and Δe_m (Equation 18) relative to the complete-$\mathbf{x}$ baseline. Importance rank (1 = most important to keep among the four ablated groups) sorts ablated modes by descending Δe_m. On PTS-CPU, ablating static hardware or dynamic `/proc` proxies increases error by ≈0.28 pp; PMU and GPU groups have smaller effects. On UnixBench all Δe_m magnitudes stay below 0.01 pp on H1. On PTS-GPU, *no_gpu* yields the largest shift ($\Delta e_m = -0.10$ pp), indicating GPU/OpenCL inventory features most strongly condition reconstruction among the ablated groups.

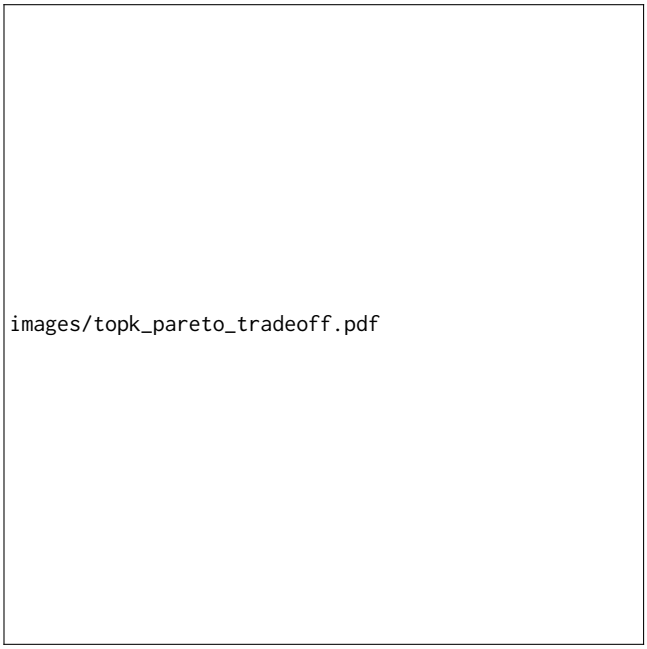


Figure 6: Top-K error–saving trade-off using the best router–reconstructor pair for each suite. Each point corresponds to a candidate K, with numbers indicating K. Stars mark the selected K according to the balanced score in Equation 17. Points closer to the lower-right corner achieve lower prediction error and higher time savings.

Table 7: System-feature ablation at fixed router policy and $K = 3$ (offline CV on H1). Δe = relative-error change vs. complete x baseline (Equation 18; pp = percentage points; baseline row omitted). Importance rank orders ablated modes by descending Δe.

Suite	Ablation mode	Rel. err. (%)	Δe (pp)	Imp. rank
UnixBench	*static_hw_only*	0.241	−0.003	2
	no_perf_pmu	0.243	0.000	1
	no_dynamic_proc	0.241	−0.003	3
	no_gpu	0.237	−0.007	4
PTS-CPU	*static_hw_only*	2.04	+0.28	1
	no_perf_pmu	1.76	0.00	3
	no_dynamic_proc	2.04	+0.28	2
	no_gpu	1.75	−0.01	4
PTS-GPU	*static_hw_only*	0.957	+0.04	1
	no_perf_pmu	0.918	0.00	3
	no_dynamic_proc	0.957	+0.04	2
	no_gpu	0.818	−0.10	4

5 RELATED WORK

5.1 System Benchmarking and Performance Evaluation

Benchmarking is a widely used methodology for evaluating the performance of operating systems [**?**]. A major line of OS benchmarking focuses on measuring low-level operating-system primitives, such as system-call overhead, context switching, process creation, interprocess communication, memory latency, cache behavior, file operations, and network performance. Representative microbenchmark suites include LMbench [**?**], UnixBench [**?**], and HBench-OS [**?**]. Brown and Seltzer [**?**] extended the LMbench methodology and introduced HBench-OS to improve benchmarking flexibility, precision, and statistical rigor. Beyond performance, Koopman et al. [**?**] proposed a robustness benchmarking method for operating systems using response regions and a CRASH severity scale to characterize erroneous system behavior. Kanoun et al. [**?**] summarized principles and examples of dependability benchmarking, emphasizing reliability, availability, serviceability, and robustness metrics for computer-system evaluation. OS benchmarking has also been adapted to domain-specific operating systems, where evaluation metrics are determined by deployment requirements. Garre et al. [**?**] compared RTOS and GPOS platforms for complex real-time simulations and showed that RTOS platforms can be preferable due to lower IPC and task-switching latency under parallel simulation workloads. Watfa et al. [**?**] proposed a benchmarking and performance evaluation approach for embedded operating systems in wireless sensor networks, highlighting the tradeoff between event-driven and thread-driven designs. Silva et al. [**?**] benchmarked IoT operating systems such as Contiki-NG, RIOT, and Zephyr using criteria including low-power consumption, real-time capability, security, interoperability, and connectivity.

5.2 Performance Prediction

Simulation-based and statistical modeling methods reduce architectural evaluation cost by simulating only selected configurations and using the results to estimate unexplored design points [**? ? ? ?**]. For example, Ipek et al. [**?**] learned predictive models from sampled architectural simulations to explore large design spaces, while Lee and Brooks [**?**] used regression models to predict performance and power from a tiny fraction of simulated microarchitectural configurations. Analytical and mechanistic models estimate performance by explicitly modeling processor behavior, such as pipeline effects, cache misses, prefetching, MSHR resources, and execution intervals [**? ? ? ? ?**]. For instance, Eyerman et al. [**?**] modeled out-of-order execution as intervals separated by disruptive miss events, and Chen and Aamodt [**?**] modeled the performance impact of pending cache hits, prefetching, and limited MSHR resources.

Machine-learning-based methods have also been widely used to learn complex relationships among programs, hardware characteristics, and performance outcomes. Ardalani et al. [**?**] proposed XAPP, which predicts GPU execution time from features of a single-threaded CPU implementation before GPU porting. Zheng et al. [**?**] introduced LACross, which predicts phase-level cross-platform performance and power traces from hardware-counter statistics collected on a host platform. Wang et al. [**?**] used deep neural

networks to predict benchmark performance for new CPUs and workloads from public CPU specifications and benchmark datasets.

6 CONCLUSION

Full-suite system benchmarking remains the gold standard for comparing platforms, yet its all-or-nothing execution model makes repeated measurement impractical in autotuning, multi-machine regression, and production-adjacent monitoring. We presented BenchScout, a lightweight pipeline that reframes suite evaluation as conditional expert selection followed by sparse reconstruction. Motivated by empirical redundancy among UnixBench subtests (Section 2.2) and early metric convergence under short runs (Section 2.2), BenchScout collects a pre-run system context $\mathbf{x}$, routes to a Top-K subset of informative components, estimates selected scores via time-bounded micro-probes, and reconstructs the full suite profile from partial observations. The design tolerates imperfect production telemetry—timeouts, missing PMU counters, and disabled eBPF tracing are encoded as features rather than treated as hard failures.

Our evaluation on five heterogeneous Linux hosts (Section 4) spans UnixBench and PTS-CPU on all machines and PTS-GPU on a GPU-equipped workstation. The primary Hybrid path (router → probe → reconstruct) recovers suite scores with mean relative errors on the order of 10^{-5}% while reducing wall time by 95–100% versus full-suite execution (Table 3), substantially outperforming execution-free predictors that generalize poorly across hosts. Resource waveforms confirm that savings come from shorter CPU-active intervals—probe-based paths finish within ≈12–48 s versus ≈1680 s for a full suite—rather than from a substantially lower memory footprint on the 4U8G and 4U16G hosts (Section 4.3). *Router only* ablations isolate router–reconstructor choices, Top-K trade-offs, and system-feature groups; *probe only* studies show that probe-based prediction remains sub-percent accurate on H1 across regressor choices (Table 6). Together, these results demonstrate that established suite scores can be approximated with minutes or seconds of observation instead of tens of minutes of full execution, without abandoning the interpretability of standard benchmark indices.

Several directions remain open. Our models are trained and evaluated per host to respect hardware heterogeneity; cross-host transfer and cold-start routing with fewer historical runs deserve further study. Hybrid accuracy is computed offline by replaying stored $\mathbf{x}$ and probe telemetry, but partial scores are always probe-model predictions before reconstruction; extending fully live hold-out sessions—where collection and inference occur in one shot without reusing stored $\mathbf{p}$—to additional suites and cloud SKUs would further strengthen deployment claims. Finally, integrating BenchScout into live autotuning loops—where probe budgets and K must adapt to search progress—is a natural next step toward making comprehensive benchmarking a routine, low-disruption component of systems research and operations.

REFERENCES

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009